Pharma Availability Control Tower MVP

UML Architecture & ERD Documentation

System Design Document

7-Day MVP Sprint

Table of Contents

1. **System Context Diagram**
2. Use Case Diagram
3. Domain / Class Diagram
4. Sequence Diagram
5. Activity Diagram — Planner Approval Flow
6. Component Diagram
7. Deployment Diagram
8. Entity Relationship Diagram (ERD)

1. System Context Diagram

The System Context Diagram positions the Control Tower MVP within its environment. It shows all external actors and data sources that interact with the platform, and clarifies the boundary between the system and its surroundings.

- External data sources: Vessel Manifest DB, Warehouse DB, DC Stock DB, Sales DB, Lorry DB
- External system: ETA Mock API refreshed every 2 hours
- Primary user: Planner — the only end-user in this MVP
- Internal subsystems: M1 Priority Scorer, M2 DC Request Generator, M3 Dispatch Planner, Demo State, Snapshot Readers
- Infrastructure: Docker-hosted MySQL 8 with named volumes on developer laptop

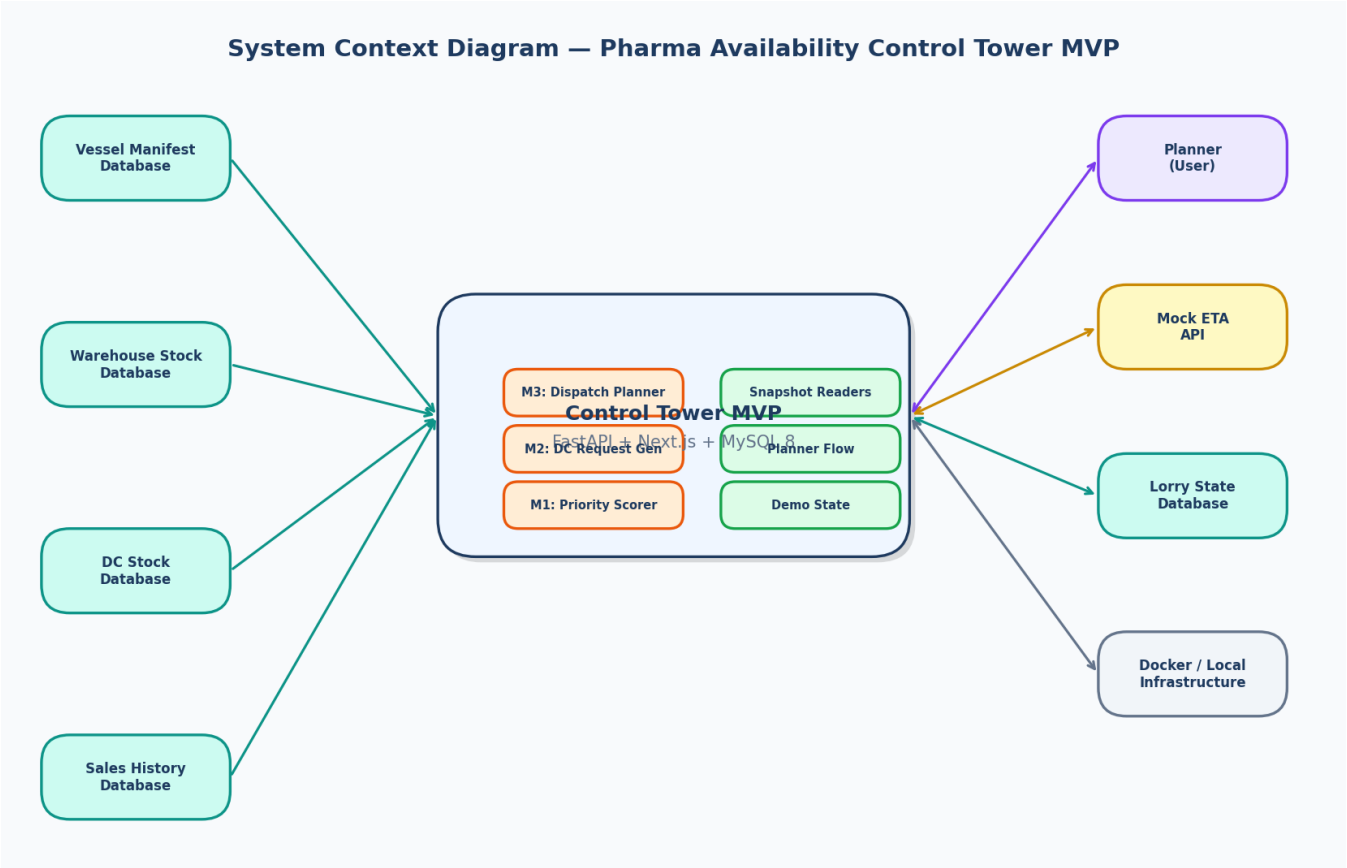

Figure 1: System Context Diagram

2. Use Case Diagram

The Use Case Diagram captures all planner interactions with the system and the two supporting system actors (ETA Mock API and DB Readers). It defines the functional scope of the MVP planner console.

- Planner use cases span dashboard, input views, M1/M2/M3 result pages, plan editing, approval, rejection, and override
- ETA Mock API feeds into ETA refresh and input snapshot use cases
- DB Readers interact with snapshot ingestion use cases
- Approval, rejection, and override are independent planner actions on draft plan versions
- History, demo-state views, and reports are read-only planner use cases

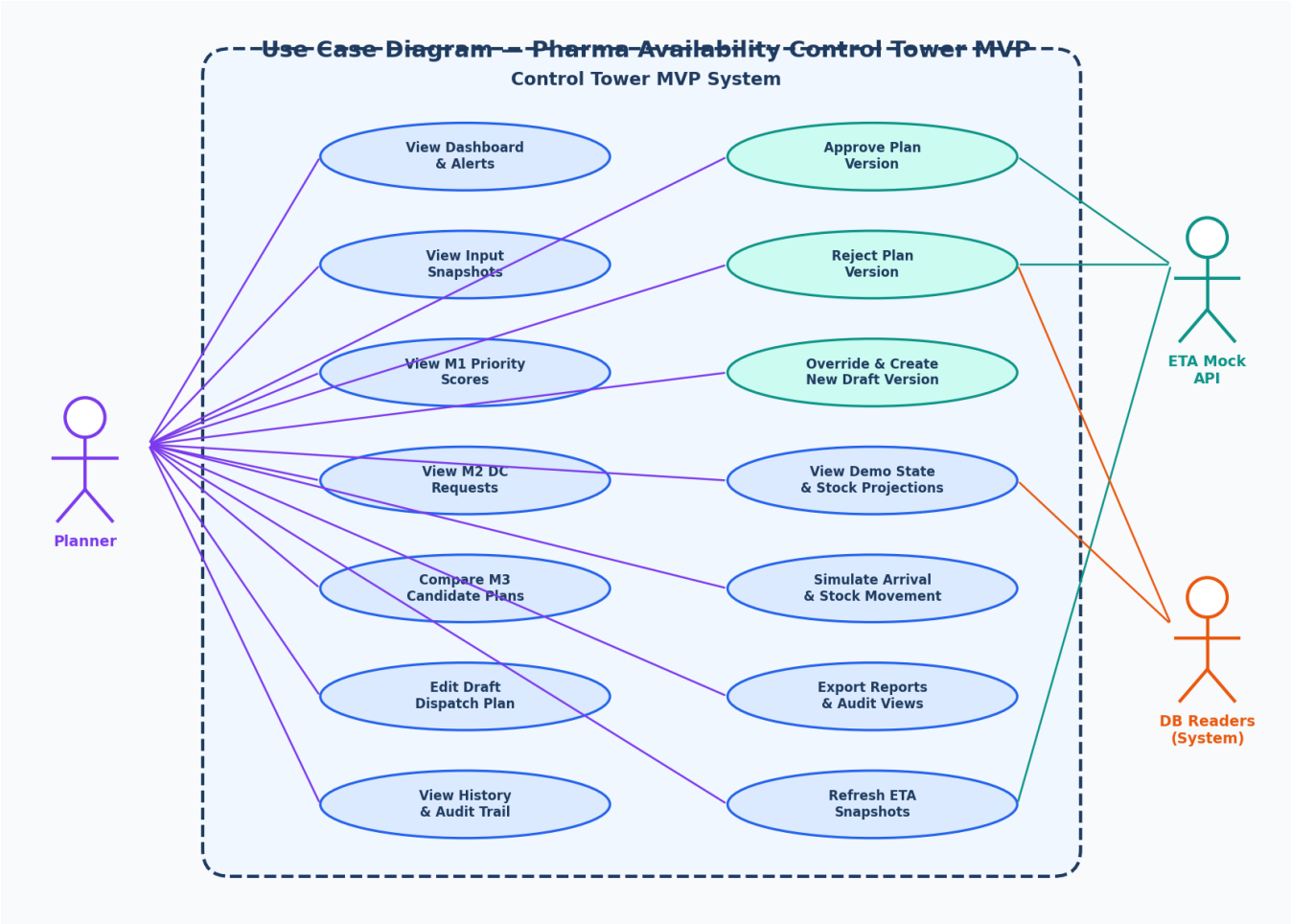

Figure 2: Use Case Diagram

3. Domain / Class Diagram

The Domain/Class Diagram models the core domain entities and their relationships. It groups entities into five cohesive layers: Master Data, Snapshots, Engine Outputs, Planner Flow, and Demo State.

- Master data: SKU, DC, Lorry, Vessel — referenced by snapshots and engine outputs
- Snapshot entities: ManifestSnapshot, ManifestLine, WarehouseStockSnapshot, DCStockSnapshot, ETASnapshot
- Engine output entities: EngineRun, M1Result, M2Request, M3PlanVersion, M3PlanStop, M3PlanItem
- Planner flow: PlannerDecision (approve/reject/override) linked to M3PlanVersion
- Demo state: DemoReservation → DemoTransfer → DemoArrivalEvent → DemoStockProjection chain
- AuditLog records all actions across entity types

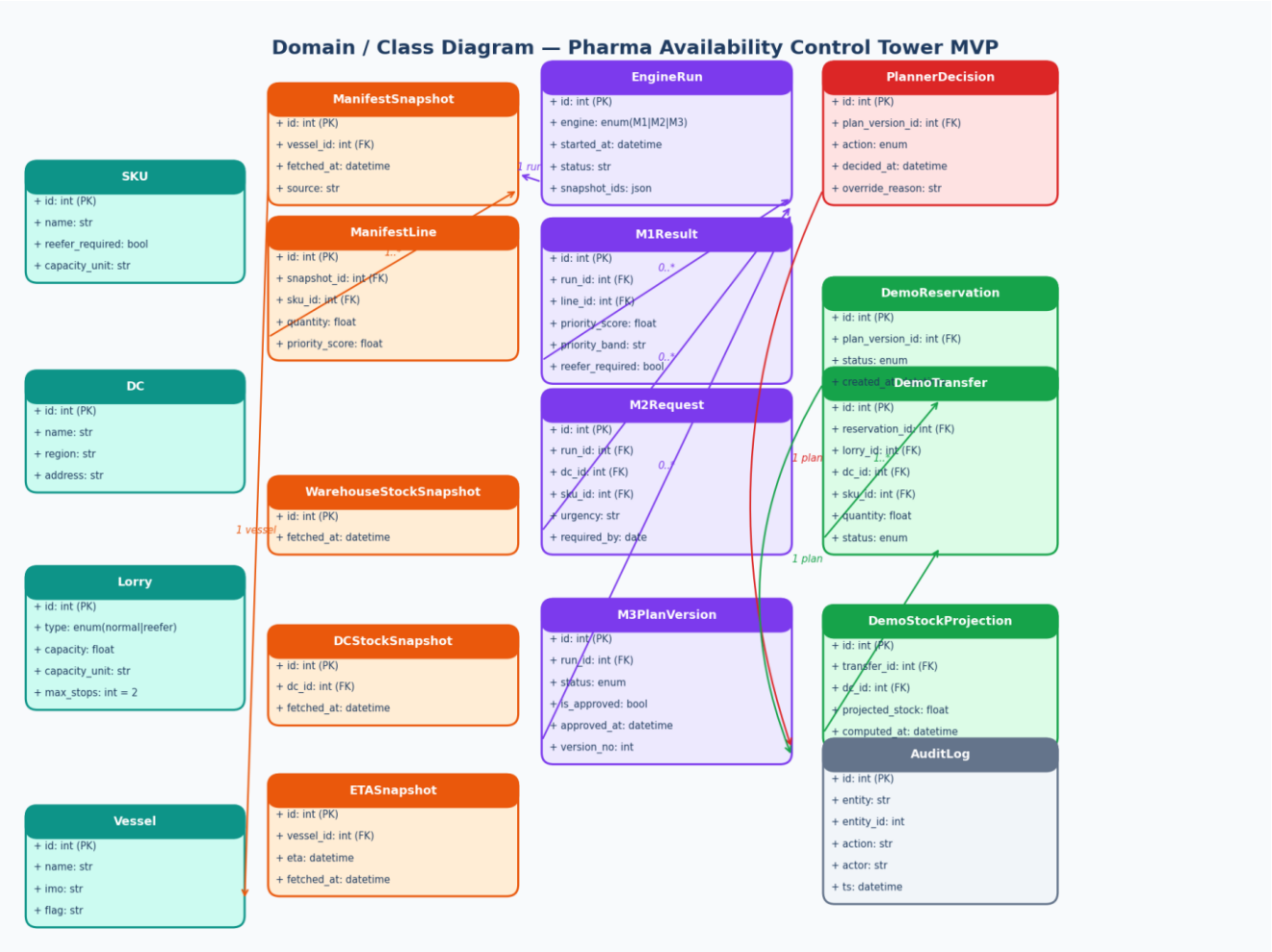

Figure 3: Domain / Class Diagram

4. Sequence Diagram

The Sequence Diagram traces the end-to-end flow from snapshot ingestion through engine execution to planner decision and demo-state simulation. It shows the temporal ordering of all inter-component messages.

- Phase 1 — Snapshot Ingestion: Manifest, Warehouse, DC, Sales, and Lorry snapshots fetched and persisted
- Phase 2 — ETA Refresh: ETA provider fetches vessel ETA and stores eta_snapshot
- Phase 3 — M2 DC Request Generation: DC stock + sales snapshot input produces dc_requests[]
- Phase 4 — M1 Priority Scoring: Manifest + stock + M2 output + ETA produces priority_scores[]
- Phase 5 — M3 Dispatch Planning: M1 + M2 + lorries + route graph produces candidate_plans[]
- Phase 6 — Planner Review: Frontend renders plan comparison for planner decision
- Phase 7 — Plan Approval: Approved plan is frozen and demo reservation + transfers are created
- Phase 8 — Arrival Simulation: Demo state processes simulated arrival and updates stock projections

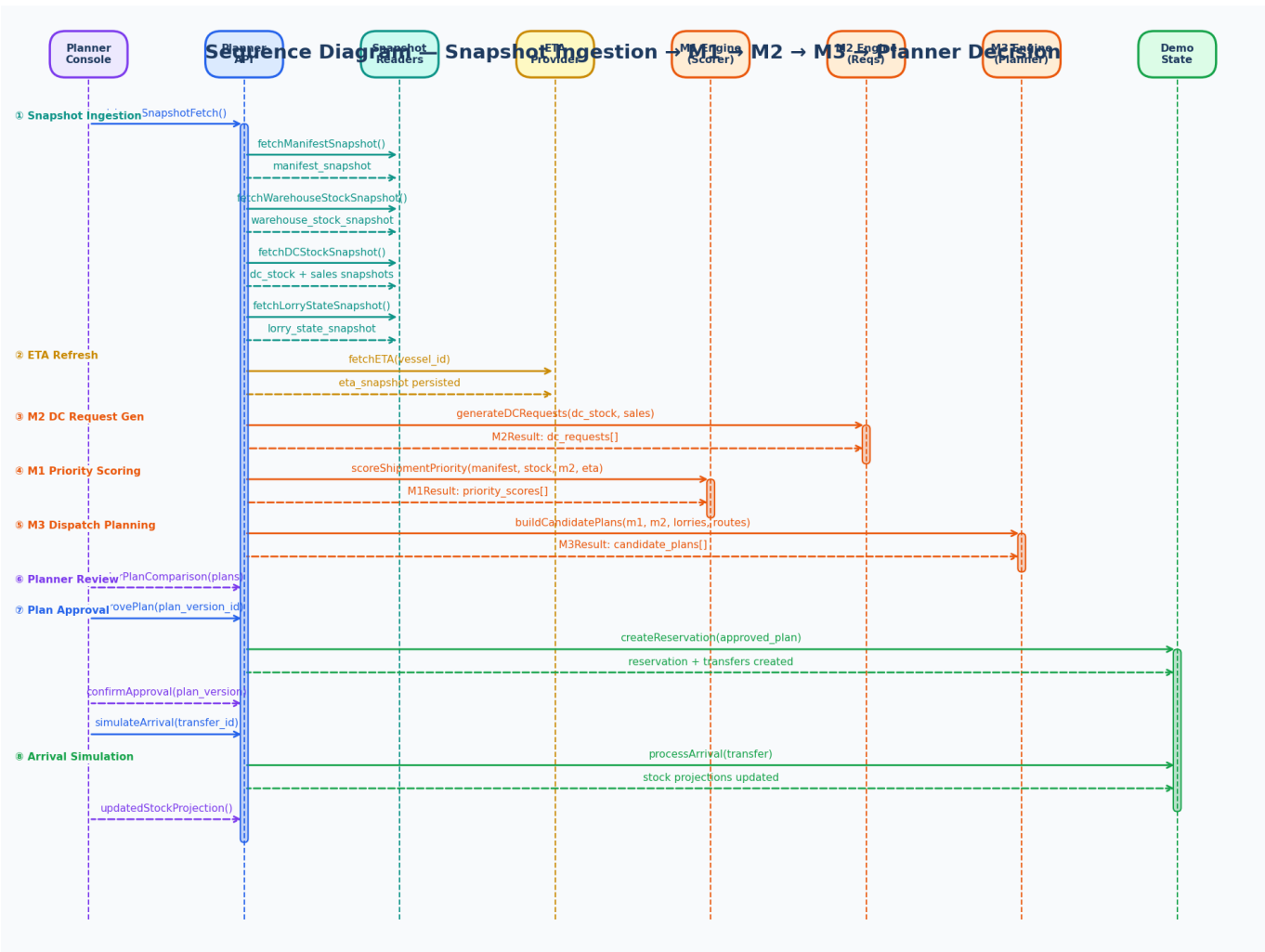

Figure 4: Sequence Diagram

5. Activity Diagram — Planner Approval Flow

The Activity Diagram models the planner's decision workflow from opening the dashboard to the final outcome: an approved (frozen) plan, a rejected plan returned to review, or a new override draft version.

- Entry point: Planner opens the Dashboard and navigates to M3 Candidate Plans
- Optional edit branch: Planner may edit lorry assignment, stop order, or quantities in the draft
- Decision gateway: Approve, Reject, or Override — three mutually exclusive outcomes
- Approve path: plan is frozen immutably, demo reservation and transfers are created, audit log is written
- Reject path: rejection is logged with reason and planner returns to plan comparison
- Override path: a new draft version is created, re-entering the review cycle

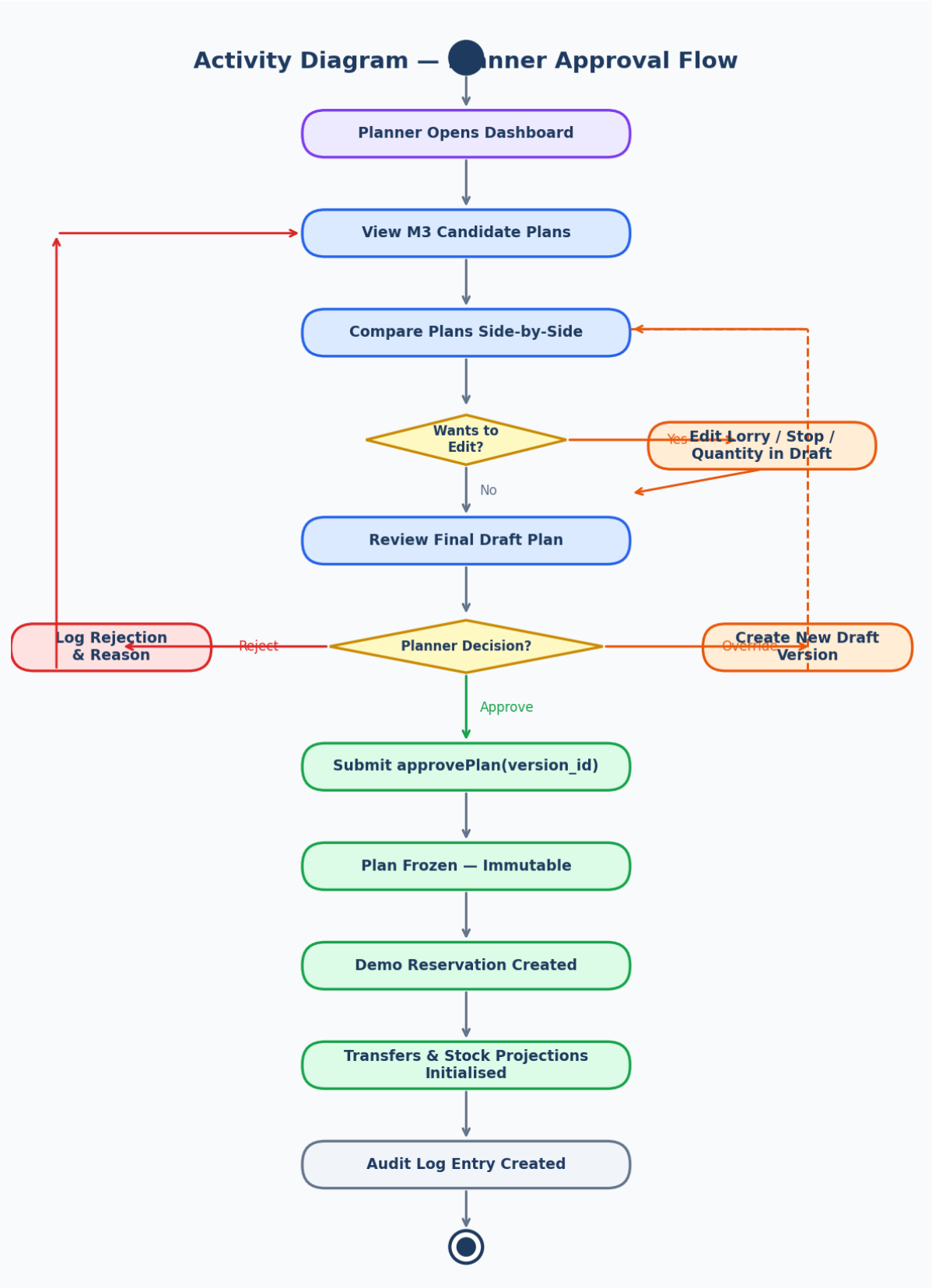

Figure 5: Activity Diagram — Planner Approval Flow

6. Component Diagram

The Component Diagram breaks the system into deployable and logical components grouped into six subsystems. It shows provided and required interfaces and inter-component dependencies.

- External Data Sources: six external readers providing raw snapshot data
- Ingestion Layer: ManifestReader, StockReaders, LorryStateReader, ETAProvider, OrchestrationService
- Persistence Layer: MySQL 8 tables managed via Alembic migrations — Snapshot, EngineRun, PlannerFlow, and DemoState table groups
- Engine Layer: M1 (XGBoost), M2 (rules-based), M3 (OR-Tools), each with a contract-compatible stub provider and a shared EngineBridge
- Demo State subsystem: ReservationService, TransferService, ArrivalSimulator, StockProjectionService
- Planner Console: Next.js pages connected via FastAPI REST layer (/api/v1/...)
- Reporting subsystem: approved-plan exports, audit views, demo-state reports, and seed/dev tools

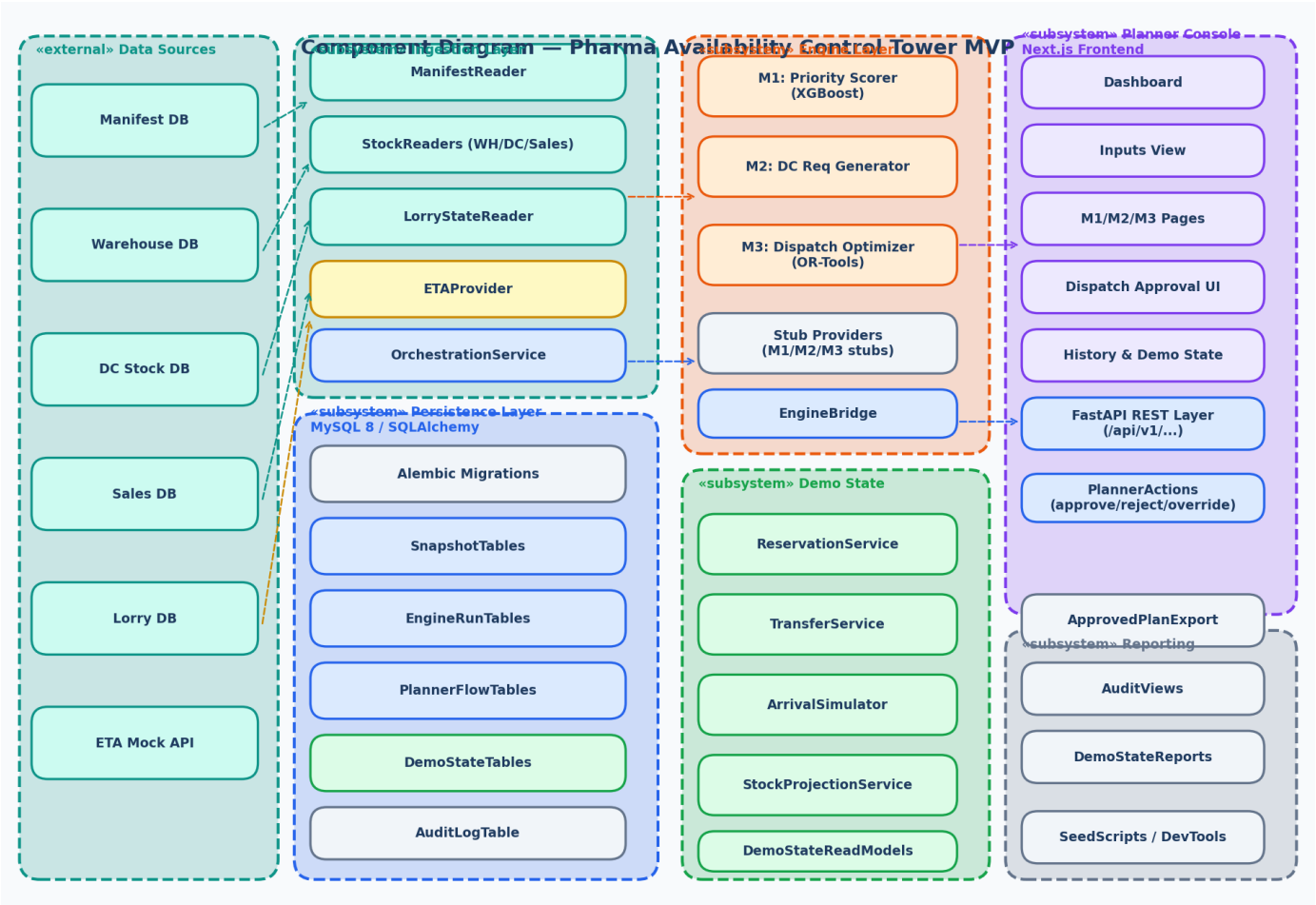

Figure 6: Component Diagram

7. Deployment Diagram

The Deployment Diagram shows how the MVP components are deployed on a single developer laptop using Docker. It models three containers — MySQL, FastAPI, and Next.js — plus the Python engine workers running within the API process.

- All containers share a Docker network named control_tower_net
- MySQL container: mysql:8 image, port 3306, named volume mysql_data for persistence
- FastAPI app container: port 8000, exposes /api/v1 REST routes for inputs, planner, demo-state, and ETA mock
- Next.js frontend container: port 3000, communicates with FastAPI over HTTP
- Python engine workers run in-process within FastAPI — M1 (XGBoost), M2, M3 (OR-Tools), with stub providers per engine
- Demo State services and Snapshot Readers also run in the FastAPI process
- ETA Mock endpoint is served by the same FastAPI app for MVP simplicity

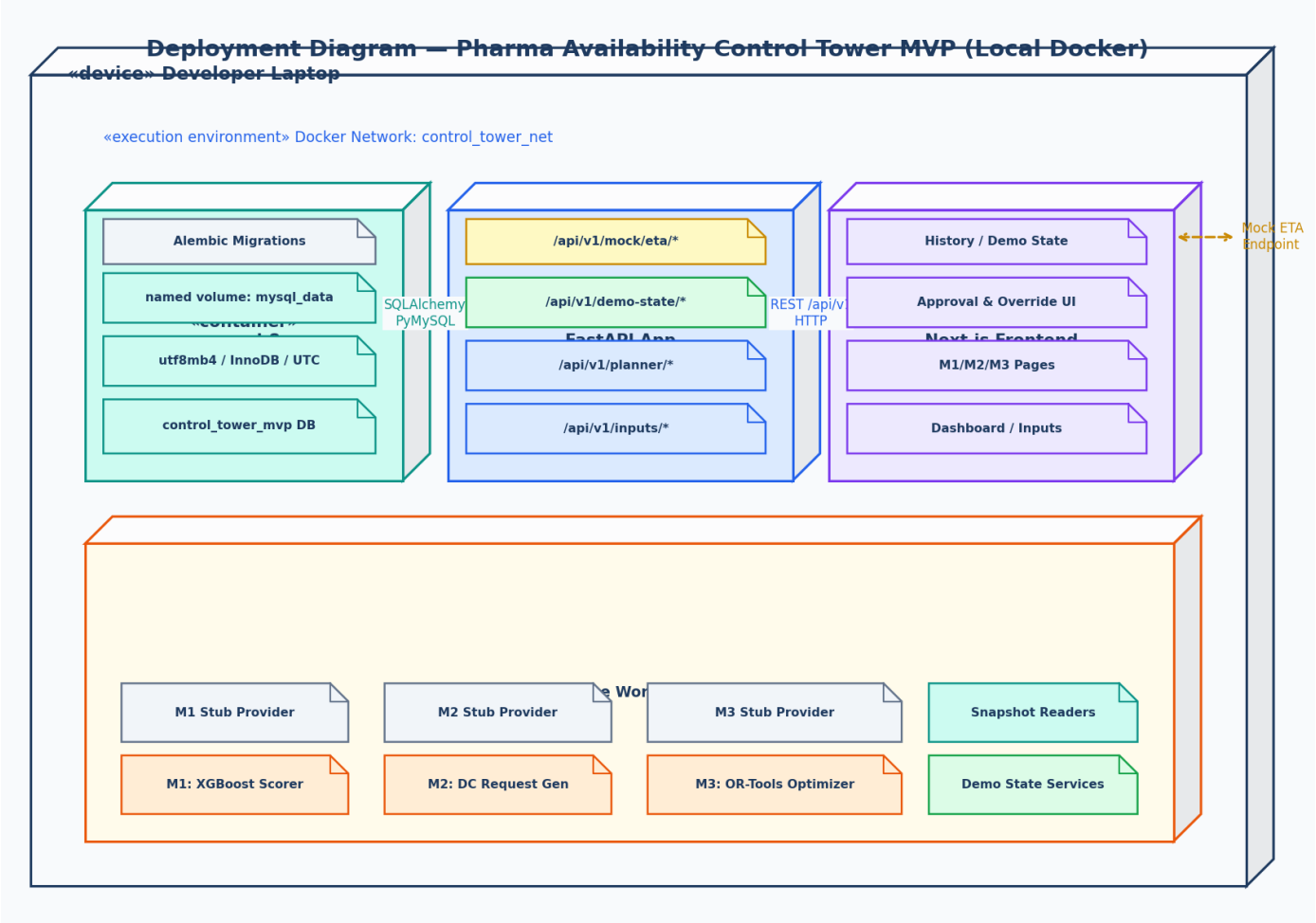

Figure 7: Deployment Diagram

8. Entity Relationship Diagram (ERD)

The ERD defines the full MySQL 8 schema for the control_tower_mvp database. Tables are grouped by domain: Master Data, Snapshots, Engine Outputs, Planner Flow, Demo State, and Audit.

- Master tables: skus, dcs, lorries, vessels, route_edges — referenced by FKs throughout
- Snapshot tables: manifest_snapshots/lines, warehouse_stock_snapshots/items, dc_stock_snapshots/items, eta_snapshots, sales_history_records, lorry_state_snapshots
- Engine tables: engine_runs (parent), m1_results, m2_requests, m3_plan_versions, m3_plan_stops, m3_plan_items
- Planner flow tables: planner_decisions, override_reasons — linked to m3_plan_versions
- Demo state tables: demo_reservations → demo_transfers → demo_arrival_events → demo_stock_projections (cascade chain)
- All tables include created_at DATETIME in UTC; approved plan versions carry approved_at and is_approved flag
- audit_logs table captures entity, entity_id, action, actor, and JSON payload for full traceability

Figure 8: Entity Relationship Diagram (ERD)